

UMDA aplicado a la Mejora Automática del Software

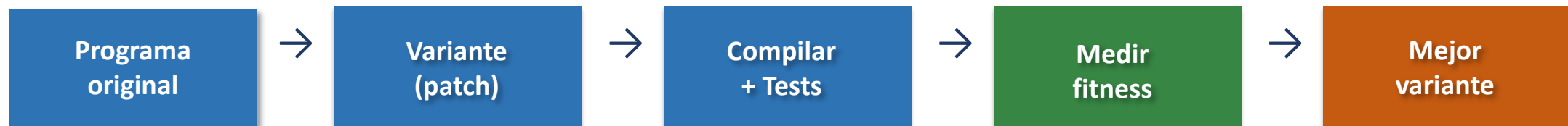
Pablo Bermejo López

Sistemas Inteligentes y Minería de Datos (SIMD)
Escuela Superior de Ingeniería Informática
Universidad de Castilla-La Mancha, Albacete

JISBD 2026 · Alicante · 16-19 de Junio de 2026

Mejora Genética del Software — ¿Por qué?

- ▶ El software contiene comportamientos no óptimos en propiedades no funcionales: tiempo de ejecución, memoria, consumo energético
- ▶ Idea: tratar el código como material genético que puede mutarse y evolucionar automáticamente (Petke et al., 2018)
- ▶ Proceso: generar variante → compilar → pasar tests funcionales → medir fitness
- ▶ Aplicaciones: reparación automática de bugs (APR), optimización de rendimiento
- ▶ **Objetivo en este trabajo: reducción del tiempo de ejecución, manteniendo la funcionalidad**

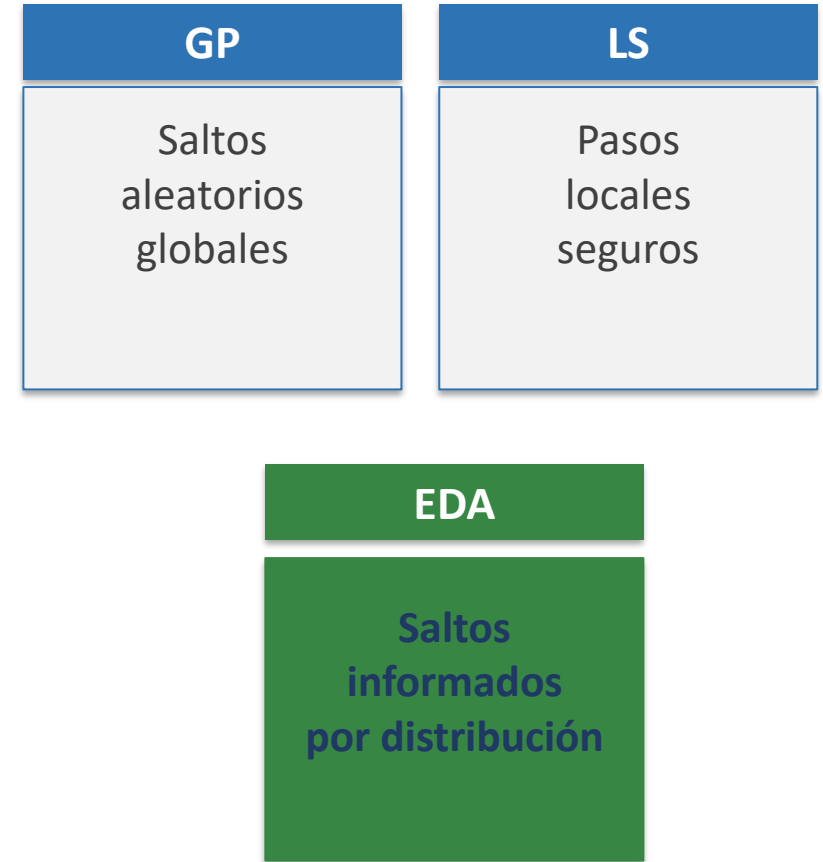


Metaheurísticas en GI

- ▶ Programación Genética (GP): estrategia dominante históricamente
→ cruces y mutaciones en el espacio de variantes
- ▶ Búsqueda Local (LS): hill climbing first-improvement
→ más efectiva y eficiente que GP (Blot & Petke, 2021, 18 estrategias)
- ▶ Búsqueda Aleatoria (RS): sorprendentemente competitiva a veces
→ evidencia la complejidad del espacio de búsqueda
- ▶ MAGPIE: framework unificado para comparar algoritmos de GI
(representación estándar de edits, cross-validation, reproducibilidad)
- ▶ **¿Existe una metaheurística robusta y general para la Mejora Genética?**

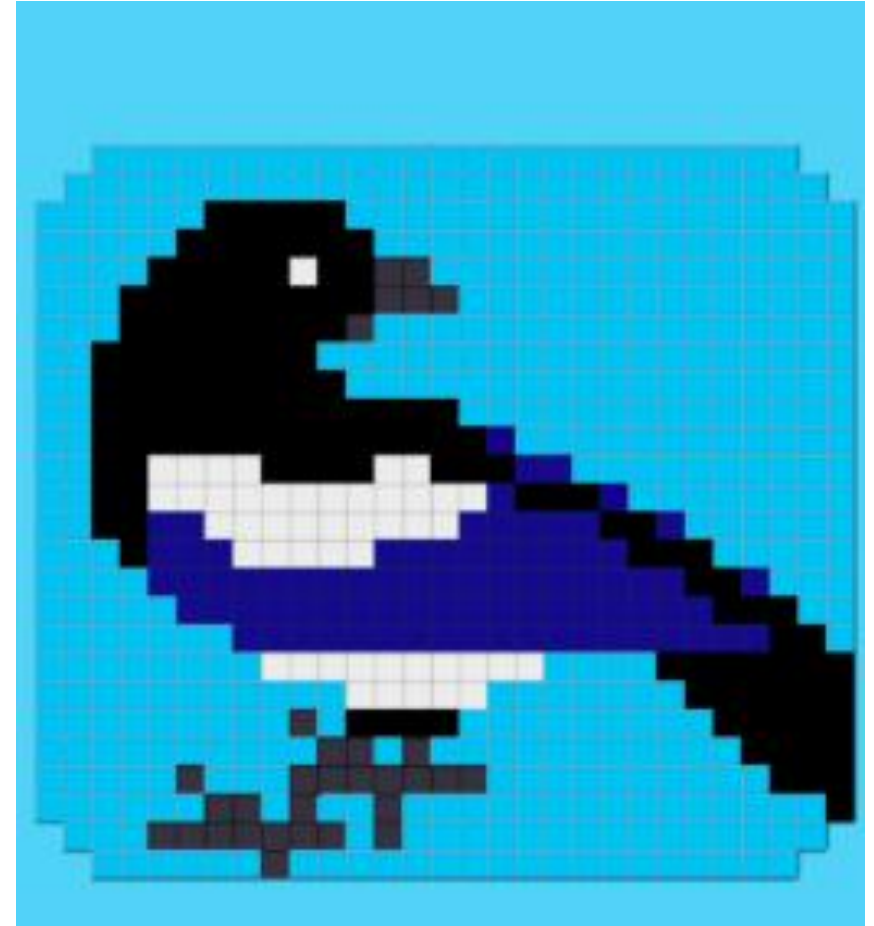
Hipótesis — ¿Por qué un Algoritmo de Estimación de Distribuciones?

- ▶ El espacio de variantes en GI está dominado por variantes inválidas (no compilan o no pasan los tests funcionales)
- ▶ GP: saltos globales ciegos → alta probabilidad de caer en zonas inválidas
- ▶ LS: avance local seguro → riesgo de quedar atrapada en mínimos locales
- ▶ **EDA (UMDA): aprende distribución marginal sobre los edits que producen variantes válidas → saltos globales informados**
- ▶ Ventaja adicional: sin hiperparámetros de cruce/mutación → configuración simple y resultados más generalizables



Framework de programación

- MAGPIE - Aymeric Blot, Justyna Petke – UCL London
- [MAGPIE: Machine Automated General Performance Improvement via Evolution of Software.](#)
- <https://github.com/bloa/magpie>



Representación del individuo en Magpie

Un edit = < Operación, Ubicación, [Ingrediente] >

Operaciones: Delete · Swap · Copy · Replace (nivel de sentencia, vía AST)

Delete s_6

Elimina sentencia en línea 6

Swap $s_3 \leftrightarrow s_4$

Intercambia líneas 3 y 4

Replace $s_2 \leftarrow s_7$

Sustituye línea 2 por línea 7

Un individuo (**parche δ**) = secuencia de edits:

Ejemplo: $\delta = (e_1, e_2)$

$e_1 = \langle \text{Swap}, s_3, s_4 \rangle \rightarrow$ intercambia sentencias en líneas 3 y 4

$e_2 = \langle \text{Delete}, s_6 \rangle \rightarrow$ elimina la sentencia en línea 6

Los individuos de una población no tienen por qué tener la misma longitud

Decisión en UMDA4GI: longitud máxima del parche $k_{\max} = 2$ edits

Parches cortos $\rightarrow$ más variantes válidas · menor sobreajuste · mejor generalización

UMDA4GI — Algoritmo

```
Inicializar  $P_0$  aleatoriamente (1 edit / individuo)
Evaluar  $P_0 \rightarrow$  identificar mejor  $\delta^*$ 

while not agotado presupuesto do
     $P_{sel} \leftarrow$  variantes válidas de  $P_i$  (compilan + tests)
    Aprender distribución  $M_i$  sobre edits en  $P_{sel}$ 
     $P_{new} \leftarrow \{ \delta^* \}$  (elitismo)
    while  $|P_{new}| < N - 1$  do
        Muestrear nuevo individuo  $\delta_{new} \sim M_i$ 
        Si  $\delta_{new} \in P_{new} \rightarrow$  sustituir por individuo aleatorio
         $P_{new} \leftarrow P_{new} \cup \{ \delta_{new} \}$ 
    end while
    Evaluar  $P_{new}$ ;
     $P_{i+1} \leftarrow P_{new}$ 
end while  $\rightarrow$  devolver mejor individuo
```

Psel dinámico

Se aprende de TODOS los individuos válidos, no un N% fijo de la población

Diversidad garantizada

Individuo duplicado $\rightarrow$ reemplazado por uno aleatorio (nuevos edits)

Sin hiperparámetros de cruce/mutación

Solo N y kmax=2 necesarios

UMDA4GI — Algoritmo

```
Aprender distribución  $M_i$  sobre edits en  $P_{sel}$   
pool  $\leftarrow$  romper todos los parches en edits individuales  
Para cada edit  
   $p(edit\_j) \leftarrow$  probabilidad marginal de cada edit único  
   $M_i \leftarrow M_i \cup p(edit\_j)$ 
```

edit único:
operación<target, Ingredient>

- Ejemplo:

Patch 1:

Edit 1: XmlNodeDeletion<stmt> (target="x = 10;")
Edit 2: XmlNodeInsertion<stmt, block> (target="if (x > 5)", ingredient="print('Greater than 5');")

Patch 2:

Edit 1: XmlNodeDeletion<stmt> (target="x = 10;")
Edit 2: XmlNodeInsertion<stmt, block> (target="while (!done)", ingredient="done = true;")

El primer edit en ambos parches aparece 2 veces

Cada Segundo edit aparece solo 1 vez.

$p[\text{XmlNodeDeletion}\langle\text{stmt}\rangle \text{ (target="x = 10;")}] \leftarrow 2/3$ (tres edits únicos)

$p[\text{XmlNodeInsertion}\langle\text{stmt, block}\rangle \text{ (target="if (x > 5)", ingredient="print('Greater than 5');")}] \leftarrow 1/3$

4 Algoritmos evolutivos comparados [pop_size=50]

- ▶ First Improvement (FI)
Búsqueda local hill-climbing
- ▶ Genetic Programming (GP)
Algoritmo genético clásico (MAGPIE default)
- ▶ Random Search (RS)
Baseline aleatoria
- ▶ **UMDA4GI ★**
Nuestra propuesta

Infraestructura HPC

Clúster galgo (SLURM) · Nodo dedicado por experimento
Métrica: wall-clock time

‘Since any specification is not fair, it is needed to use various termination conditions’

‘it is not likely that a single algorithm is always the best over a wide range of generations,

[Ishibuchi22]

5 Presupuestos (max evals)

100 · 250 · 500 · 750 · 1000

4 aplicaciones reales de código abierto · 3 lenguajes · dominios distintos

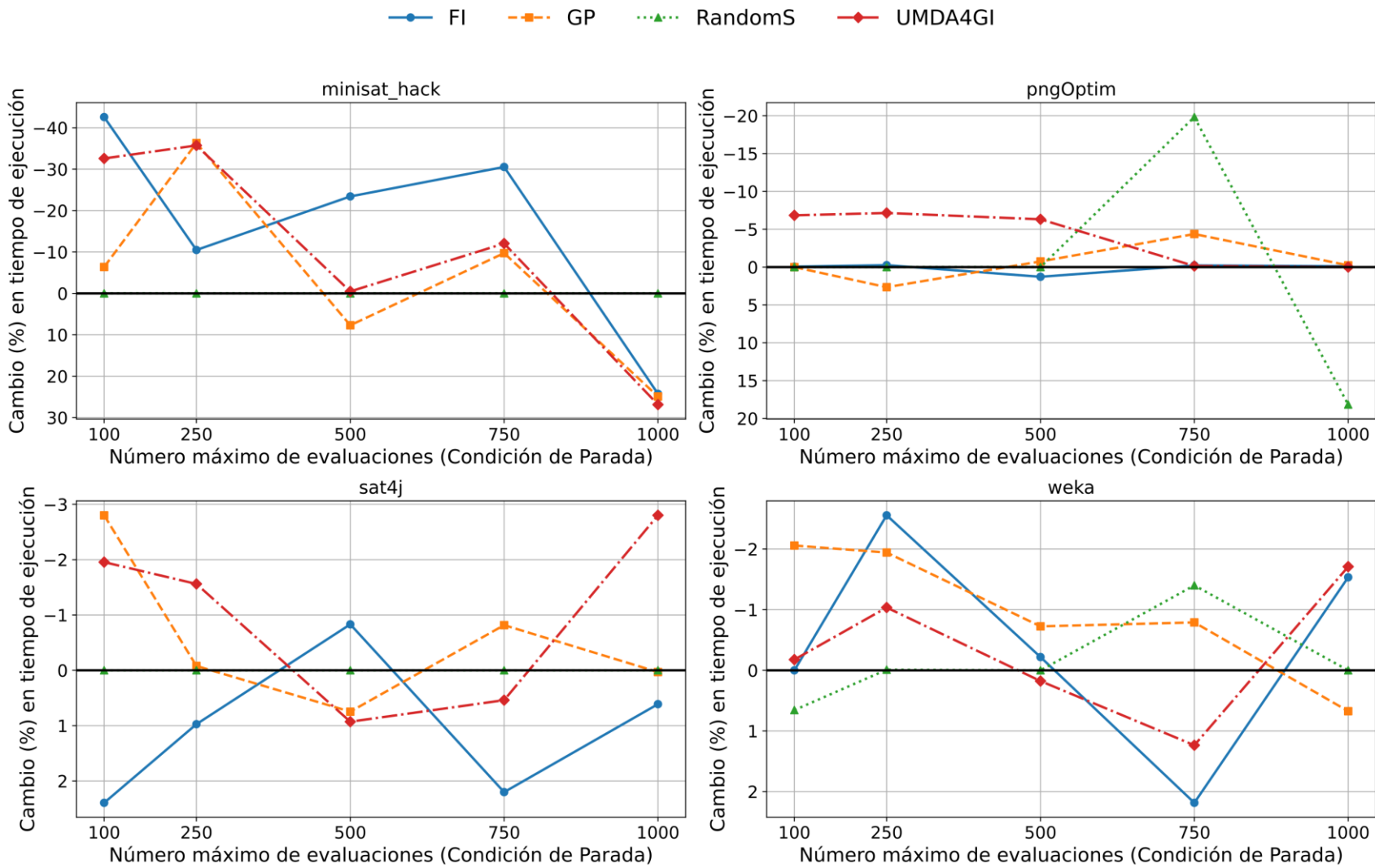
MiniSAT-hack	Sat4J	Weka	OptimPNG
Lenguaje: C++	Lenguaje: Java	Lenguaje: Java	Lenguaje: C
Dominio: SAT Solving	Dominio: SAT Solving	Dominio: Machine Learning (Random Forest)	Dominio: Optimización PNG
Objetivo: Solver.cc	Objetivo: (múltiples clases)	Objetivo: RandomForest.java	Objetivo: optipng.c, optmi.c
Datos: 20 archivos .cnf 50–200 variables	Datos: Mismo corpus que MiniSAT-hack	Datos: 20 datasets .arff (adjuntos al SW)	Datos: 29 imágenes PNG (color, repositorio ImageMagick)

Validación cruzada

5-fold cross-validation

Total: 4 alg × 4 sw × 5 budgets × 5 folds = 400 experimentos

Resultados I — Reducción de tiempo de ejecución (validación)



Ranking global (promedio sobre todos los presupuestos):

Algoritmo	minisat	sat4j	weka	pngOptim
First Imp.	1	4	2	4
Gen. Prog.	3	2	1	2
Rand. S.	4	3	4	3
UMDA4GI	2	1	3	1

UMDA4GI: único algoritmo con 2 primeros puestos (sat4j y pngOptim)

↑ Menor es mejor (reducción %) · Eje Y invertido en figura

Resultados II — Sobreajuste (overfitting)

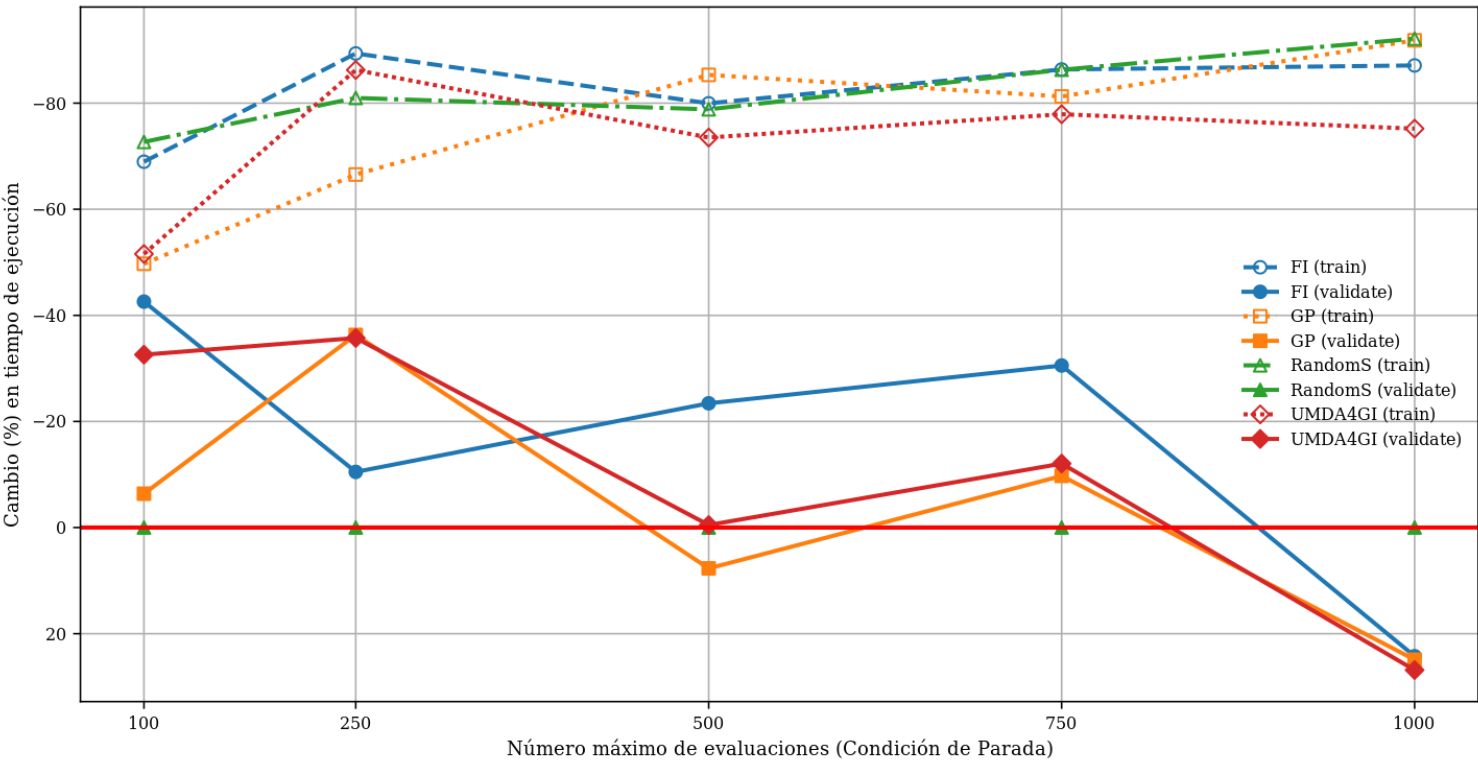


Gráfico con minisat_hack
Líneas sólidas = validación · Líneas discontinuas = entrenamiento

Ranking — menor sobreajuste

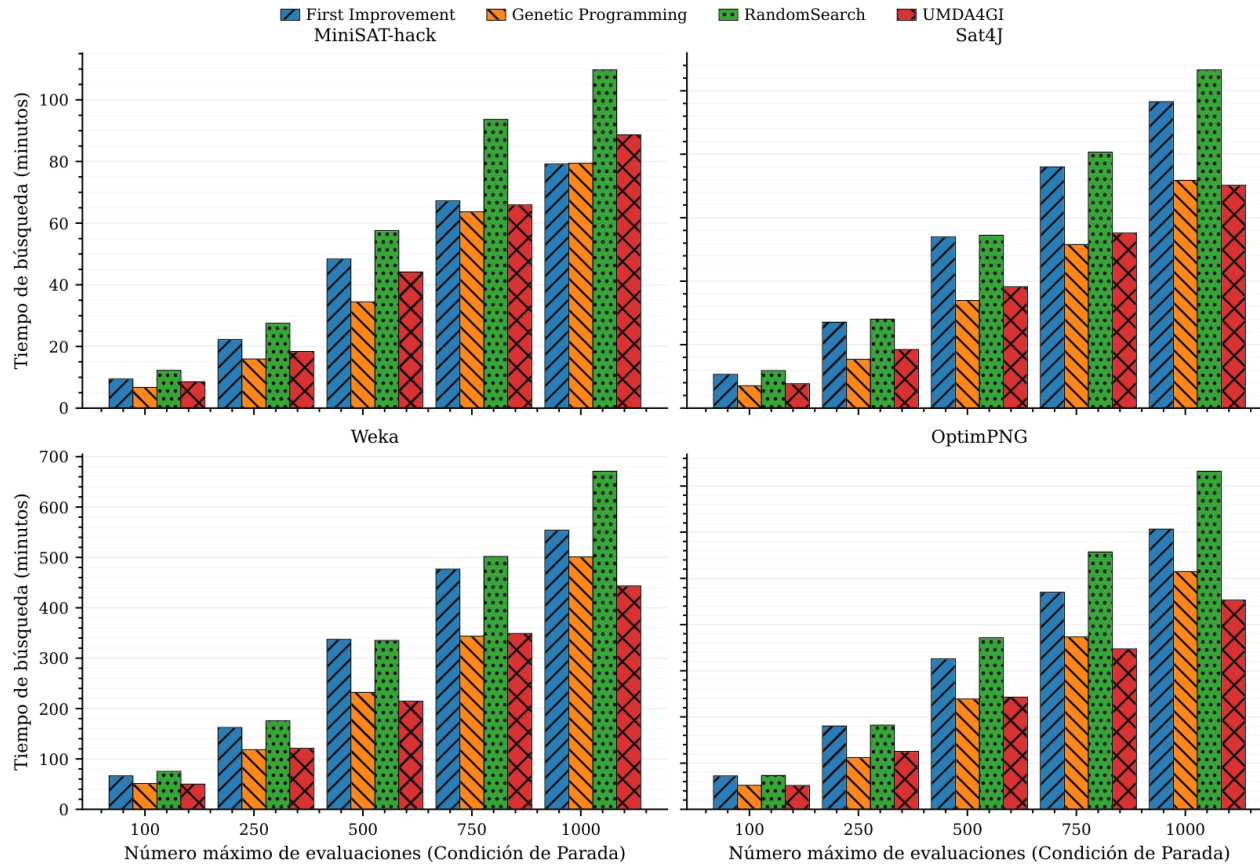
Algoritmo	minisat	sat4j	weka	pngOptim
First Imp.	2	3	3	3
Gen. Prog.	3	2	1	2
UMDA4GI	1	1	2	1

UMDA4GI: menor sobreajuste
en 3 de 4 softwares
(2º en Weka)

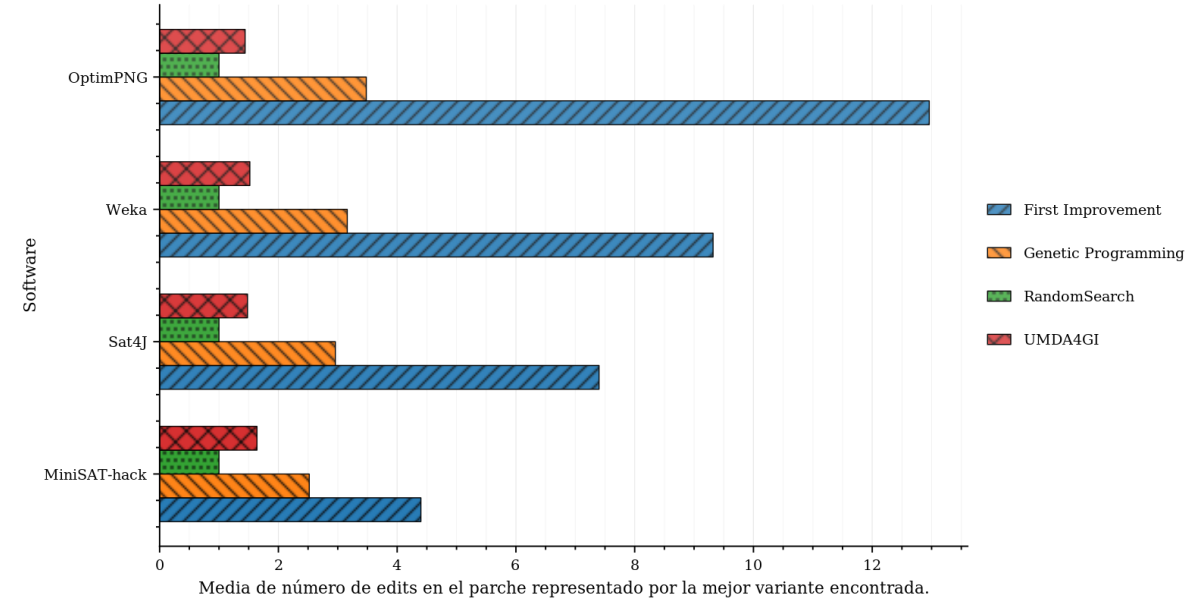
Causa probable: kmax=2 edits
→ parches cortos generalizan mejor

Resultados III — Tiempo de búsqueda y longitud de parches

Tiempo de búsqueda (minutos):



Longitud media del parche (nº de edits):



- ▶ UMDA4GI y GP: los algoritmos más rápidos
- ▶ Parches cortos (kmax=2) → el menor número de edits entre los algoritmos poblacionales
- ▶ Parches más largos (FI) → más sobreajuste (consistente con Petke et al., 2023)

Conclusiones

1

FITNESS COMPETITIVO

UMDA4GI obtiene los mejores resultados en 2 de 4 softwares (sat4j, pngOptim) y posiciones competitivas en los otros dos.

Es el único algoritmo con dos primeros puestos en el ranking global.

2

MENOR SOBREAJUSTE

UMDA4GI generaliza mejor que FI y GP en 3 de 4 softwares.

Los parches de 2 edits reducen el overfitting al no sobreajustar la búsqueda a los datos de entrenamiento.

3

VELOCIDAD INTERMEDIA

Tiempo de búsqueda similar a GP, sin hyperparámetros de cruce/mutación: Fácil de configurar y reproducir.

Trabajo futuro

→ Filtrado de Psel

Aprender la distribución solo sobre variantes que mejoran el fitness
(no sobre todas las válidas) → ¿distribución de mejora más pura?



LLMs como operadores (en progreso)

Usar modelos de lenguaje para generar edits semánticamente informados.
UMDA4GI + LLM (UMDA4GI): LLM aprende de las mejores variantes → genera los edits de la siguiente generación guiándose por el código y el historial.



LLM puro como baseline

¿Puede un LLM solo (sin EDA) mejorar el software?
Experimentos en ejecución actualmente para aislar la contribución del LLM.

→ Análisis semántico de la distribución

¿Qué aprende UMDA sobre el código? ¿Hay edits recurrentes con significado?
Interpretabilidad del modelo probabilístico aprendido.

¡Gracias! / ¿Preguntas?

Email

pablo.bermejo@uclm.es

Código

github.com/paberlo/magpie

Datos

zenodo.org/records/19469242

UMDA aplicado a la Mejora Automática del Software · JISBD 2026 · Alicante